作业 __Week12

20244227004 蔡博凯

2024 年 11 月 26 日

第一题

ExoticNet的输入为100X100X3的图像，输出为100个类别的条件概率，其具体结构如下：
第一：卷积层，使用64个4X4X3的卷积核，stride=2，padding=2(边缘上填0的个数）；
第二：Pooling层，使用5X5的max pooling，stride=5；
第三：卷积层，使用6400个10X10X64的卷积核，stride=1，padding=0；
第四至六：共三层全连层，神经元数量分别为1600、400和100。
请问这个网络的参数量。

解答

1. 首先获得各参数的计算公式

1.1 卷积层输出张量计算公式

O = 输出图像的大小（此处指宽度）

I = 输入图像的大小（此处指宽度）

K = 卷积层中使用的卷积核的大小（此处指宽度）

N = 卷积核个数。

S = 卷积运算的步长。

P = 填充个数。

输出图像的大小（O）由下式确定：

$$O = \frac{I - K + 2P}{S} + 1 \tag{1.1}$$

输出图像中的通道数等于该卷积层的卷积核个数 N。

1.2 池化层输出张量计算公式

O = 输出图像的大小（此处指宽度）

I = 输入图像的大小（此处指宽度）

S = 卷积运算的步长

P_s = 池大小

输出图像的大小（O）由下式确定

$$O = \frac{I - P_s}{S} + 1 \tag{1.2}$$

1.3 完全连接层输出张量

一个完全连接层输出一个向量，该向量长度（向量的元素个数）等于层中的神经元个数。

1.4 卷积层的参数个数

在 CNN 中，每一层都有两种参数：权重和偏差。参数的总数只是所有权重和偏差的总和。

W_c = Conv 层的权重个数

B_c = Conv 层的偏差个数

P_c = Conv 层的参数个数

K = 卷积层中使用的卷积核的大小（此处指宽度）

N = 卷积核个数

C = 该层输入图像的通道数

$$W_c = K^2 \times C \times N \tag{1.3}$$

$$B_c = N \tag{1.4}$$

$$P_c = W_c + B_c \tag{1.5}$$

1.5 池化层的参数个数

没有与最大池化层相关联的参数。池的大小、步长和填充都是超参数。

1.6 全连接（FC）层的参数个数

在 CNN 中有两种完全连接的层。第一种是该全连接层的前面连接着最后

一个卷积层，第二种是该全连接层的前面连接着其他全连接层。让我们分别考虑每一种情况。

情况 1：前面连接着卷积层的全连接（FC）层的参数个数

W_{cf} = 连接到卷积层的全连接层的权重个数

B_{cf} = 连接到卷积层的全连接层的偏差个数

O = 前一个卷积层的输出图像的大小（宽度）

N = 前一个卷积层中的卷积核个数

F = 全连接层中的神经元个数

$$W_{cf} = O^2 \times N \times F \tag{1.6}$$

$$B_{cf} = F \tag{1.7}$$

$$P_{cf} = W_{cf} + B_{cf} \tag{1.8}$$

情况 2：前面连接着其它全连接层的全连接（FC）层的参数个数

W_{ff} = 全连接层的权数

B_{ff} = 全连接层的偏差数

P_{ff} = 全连接层的参数数目

F = 全连接层中的神经元数

F_{-1} = 前一个全连接层中的神经元数

$$W_{ff} = F_{-1} \times F \tag{1.9}$$

$$B_{ff} = F \tag{1.10}$$

$$P_{ff} = W_{ff} + B_{ff} \tag{1.11}$$

2 根据以上公式计算参数量

Layer Name	Tensor Size	Weights	Biases	Parameters
Input Image	100x100x3	0	0	0
Conv-1	51x51x64	3072	64	3136
Max Pool	10x10x64	0	0	0
Conv-2	1x1x6400	40960000	6400	40966400
FC-1	1600x1	10240000	1600	10241600
FC-2	400x1	640000	400	640400
FC-3	100x1	40000	100	40100
Output	100x1	0	0	0
Total				51891636

表 1: 参数计算结果

3 代码验证

使用代码模拟上述计算过程，得出结果如下所示，该结果证明计算的正确性。代码详情请参考cnnparams.

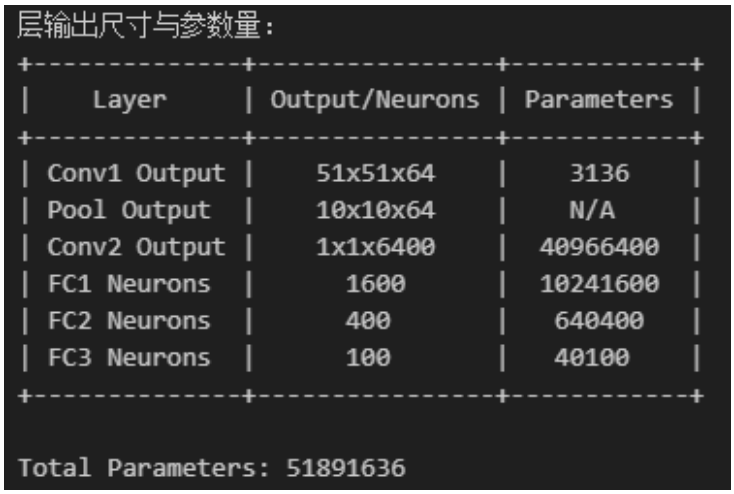
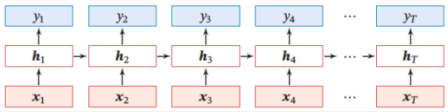


图 1: 运行结果

第二题

Simple RNN可以写为 $\boldsymbol{h}_t = f(\boldsymbol{U}\boldsymbol{h}_{t-1} + \boldsymbol{W}\boldsymbol{x}_t + \boldsymbol{b})$ （见下图），但它会受到梯度消失或爆炸的困扰。有一种改进方法是将其迭代的方式改作 $\boldsymbol{h}_t = \boldsymbol{h}_{t-1} + g(\boldsymbol{x}_t, \boldsymbol{h}_{t-1}; \theta)$，请分析它为何可以缓解梯度消失的问题，以及它是否仍然存在问题？



解答

在标准 RNN 中，隐状态更新公式为：

$$h_t = f(Uh_{t-1} + Wx_t + b)$$

其中，f 是非线性激活函数。这个公式会受到梯度消失或梯度爆炸的问题困扰，尤其在长时间序列中，梯度在反向传播过程中会逐渐变小（梯度消失）或增大（梯度爆炸）。

梯度消失的缓解

在经典 RNN 中，隐状态的更新通过多次矩阵乘法累积，若权重矩阵的范数小于 1，梯度在反向传播时会逐渐消失。而在改进后的 RNN 中，隐状态的更新为：

$$h_t = h_{t-1} + g(x_t, h_{t-1}; \theta)$$

这一更新形式使得隐状态更新不再依赖于前一时刻状态的乘积累积，从而减少了梯度衰减的效果，缓解了梯度消失的问题。

可能的挑战

尽管该改进能够缓解梯度消失问题，但仍然可能面临以下挑战：

- **梯度爆炸：** 如果增量函数 $g(x_t, h_{t-1}; \theta)$ 的权重过大，仍然可能导致梯度爆炸，特别是在处理长序列时。
- **长期依赖问题：** 尽管改进后的方法能够缓解梯度消失，但在极长序列中，依然难以捕捉到远距离的依赖关系。

- **计算复杂度：**为了增强表达能力，增量函数可能需要更复杂的结构，这会增加计算复杂度。

第三题

请写出LSTM和GRU的全套公式和结构图，并比较两者的异同。试分析相对于原始RNN，为何门控结构可以缓解梯度消失的问题。

解答

1. LSTM（长短期记忆网络）

LSTM 通过引入门控机制来控制信息流动，包括三个门：遗忘门（Forget Gate）、输入门（Input Gate）和输出门（Output Gate）。

LSTM 的状态更新过程如下：

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad \text{（遗忘门）} \tag{1}$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad \text{（输入门）} \tag{2}$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad \text{（候选细胞状态）} \tag{3}$$

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t \quad \text{（更新细胞状态）} \tag{4}$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad \text{（输出门）} \tag{5}$$

$$h_t = o_t \cdot \tanh(C_t) \quad \text{（隐状态）} \tag{6}$$

2. GRU（门控循环单元）

GRU 具有较少的门控机制，主要通过更新门（Update Gate）和重置门（Reset Gate）来控制信息的流动。

GRU 的状态更新过程如下：

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z) \quad \text{(更新门)} \tag{7}$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r) \quad \text{(重置门)} \tag{8}$$

$$\tilde{h}_t = \tanh(W_h \cdot [r_t \cdot h_{t-1}, x_t] + b_h) \quad \text{(候选隐状态)} \tag{9}$$

$$h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot \tilde{h}_t \quad \text{(隐状态更新)} \tag{10}$$

3. LSTM 与 GRU 的结构图

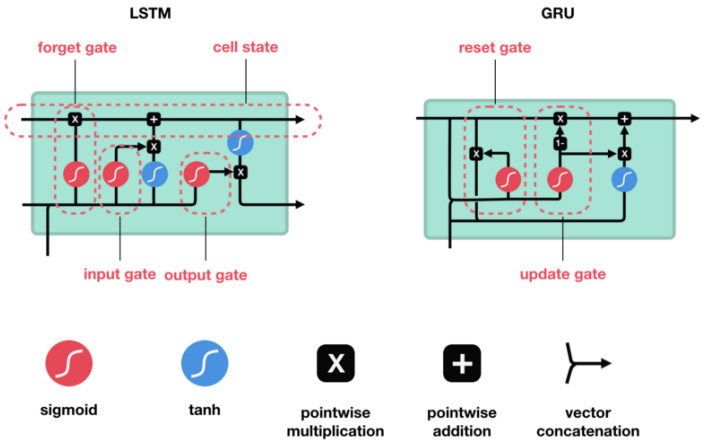


图 2: LSTM 与 GRU 的结构图

4. LSTM 与 GRU 的异同

- **相同点：** LSTM 和 GRU 都用于处理长时间序列数据，通过门控机制控制信息流动，有效缓解了梯度消失问题。
- **不同点：**
  - LSTM 使用三个门（遗忘门、输入门和输出门），而 GRU 使用两个门（更新门和重置门）。
  - LSTM 的细胞状态和隐状态是分开的，而 GRU 将细胞状态和隐状态合并。

– LSTM 计算复杂度较高，而 GRU 相对较为简洁，计算效率较高。

5. 门控结构如何缓解梯度消失问题

LSTM 和 GRU 的门控机制通过选择性地更新和传递信息，缓解了传统 RNN 的梯度消失问题：

- **控制信息流动：** LSTM 通过遗忘门决定保留多少历史信息，输入门控制新增的信息量，输出门决定哪些信息传递给下一时刻。GRU 通过更新门控制隐状态更新，通过重置门控制历史信息的遗忘。
- **保持长期依赖：** LSTM 通过细胞状态（Cell State）使得信息能够在多个时间步长中持续传递，避免信息过早消失。GRU 通过更新门帮助保留重要的历史信息，并控制当前信息的引入。
- 这些门控结构帮助减轻了反向传播中的梯度消失问题，使得网络能够有效处理长时间依赖的序列数据。

第四题

请分析缩放点积模型 $s(\boldsymbol{x},\boldsymbol{q})=\dfrac{\boldsymbol{x}^{\mathrm{T}}\boldsymbol{q}}{\sqrt{D}}$ 可以缓解Softmax函数梯度消失的原因。

解答

分析缩放点积模型的作用

缩放点积模型用于计算查询向量 x 和键向量 q 之间的相似度，其计算公式为：

$$s(x,q)=\frac{x^{T}q}{\sqrt{D}}$$

其中，$x^{T}q$ 是查询向量和键向量的点积，D 是查询和键向量的维度。引入缩放因子 $\sqrt{D}$ 的目的是缓解 Softmax 函数在计算注意力权重时可能遇到的梯度消失问题。

Softmax 函数的梯度消失问题

Softmax 函数常用于将输入的实数转化为概率分布。在注意力机制中，Softmax 用于计算查询和键之间相似度的加权概率。但是，当点积 x^Tq 的值非常大或非常小时，Softmax 的输出会变得非常接近 0 或 1，从而导致梯度接近零，这被称为梯度消失问题。具体地，如果 x^Tq 的绝对值非常大，Softmax 的输出将非常不均匀（接近 0 或 1），此时 Softmax 的梯度将变得非常小，影响反向传播和权重更新。

缩放点积的引入

为了避免上述问题，引入了缩放因子 $\sqrt{D}$，将点积结果进行规范化。具体公式为：

$$s(x,q) = \frac{x^Tq}{\sqrt{D}}$$

当查询和键向量的维度 D 增加时，x^Tq 的值可能会增大。如果不进行缩放，点积的绝对值可能会变得非常大，导致 Softmax 的梯度非常小。而通过缩放，我们将点积的结果限制在一个较合理的范围内，避免了梯度消失的问题。

缓解梯度消失的原因

缩放因子 $\sqrt{D}$ 的引入有效地平衡了点积的结果，防止其过大。当维度 D 较大时，点积的结果 x^Tq 会随着维度的增加而增大。如果不进行缩放，Softmax 函数的输入可能会变得非常大，从而导致输出的概率分布过于平坦，导致梯度接近零。通过引入 $\sqrt{D}$ 进行缩放，我们将点积的范围缩小，使得 Softmax 的输出概率分布保持在一个较为平衡的范围内，从而避免了梯度消失。

结论

缩放点积模型通过引入 $\sqrt{D}$ 缩放因子，减小了点积的值，从而防止了 Softmax 输出概率分布过于平坦的问题。这帮助保持了有效的梯度流，使得模型在训练过程中能够有效地更新权重并收敛。

第五题

请写出用Pointer Network求解Traveling Salesman Problem的伪代码，并比较此模型与无注意力机制的序列模型在求解这个问题上的优劣。

解答

1. Pointer Network for TSP

给定一组城市的坐标 $X = \{x_1, x_2, \ldots, x_n\}$，目标是找到一条最优路径，访问每个城市一次并返回起点。

Algorithm 1 Pointer Network 求解 TSP

输入: 城市坐标 $X = \{x_1, x_2, \ldots, x_n\}$
输出: 城市访问顺序 $\pi = [\pi_1, \pi_2, \ldots, \pi_n]$
初始化编码器输出和状态: $(h_1, h_2, \ldots, h_n), s_0 = \text{Encoder}(X)$
初始化解码器状态: $s_{\text{decoder}} = s_0$
初始化已访问城市列表: visited_cities = []
for t = 1 to n **do**
    计算注意力权重: $\alpha_t = \text{Attention}(h_1, h_2, \ldots, h_n, s_{\text{decoder}})$
    选择下一个城市: $\pi_t = \text{PointerSelection}(\alpha_t)$
    将 π_t 添加到已访问城市: visited_cities.append(π_t)
    更新解码器状态: $s_{\text{decoder}} = \text{UpdateDecoderState}(s_{\text{decoder}}, \pi_t)$
    屏蔽已访问的城市: $(h_1, h_2, \ldots, h_n) = \text{MaskVisitedCities}$
end for
返回访问顺序 visited_cities

2. Pointer Network 与无注意力机制序列模型的比较

- **精度：** Pointer Network 通常能获得更高的精度，因为它能够在解的生成过程中进行动态调整，从而找到更优的路径。
- **可解释性：** Pointer Network 更易于理解和解释，因为它直接选择位置，而传统模型则较难解释每一步的决策。
- **训练和推理效率：** Pointer Network 通过注意力机制可以在每一步选择最优路径，不需要显式生成整个序列，而无注意力机制的模型则可能需要更多时间来收敛。

总之，Pointer Network 在求解像 TSP 这样的排列优化问题时，比无注意力机制的传统序列模型更具优势，能够提供更高的求解精度和灵活性。

参考文献

[1] https://www.cnblogs.com/MinPage/p/14088826.html

A cnnparams

```python
import numpy as np
from prettytable import PrettyTable

# 输入尺寸
input_shape = (100, 100, 3)
# 1. 第一层：卷积层
conv1_filters = 64
conv1_kernel_size = (4, 4, 3)  # 高 x 宽 x 深度
conv1_stride = 2
conv1_padding = 2
# 输出尺寸计算公式：(输入尺寸 + 2 * padding - kernel_size) // stride + 1
conv1_output_height = (input_shape[0] + 2 * conv1_padding - conv1_kernel_size[0]) // conv1_stride + 1
conv1_output_width = (input_shape[1] + 2 * conv1_padding - conv1_kernel_size[1]) // conv1_stride + 1
conv1_output_depth = conv1_filters
conv1_params = np.prod(conv1_kernel_size) * conv1_filters  # 卷积核参数量
conv1_bias = conv1_filters  # 每个卷积核有一个偏置
conv1_total_params = conv1_params + conv1_bias
# 2. 第二层：Pooling层
```

```
19 pool_kernel_size = 5
20 pool_stride = 5
21 pool_output_height = (conv1_output_height - pool_kernel_size) //
       pool_stride + 1
22 pool_output_width = (conv1_output_width - pool_kernel_size) //
      pool_stride + 1
23 pool_output_depth = conv1_output_depth
24 # 3. 第三层: 卷积层
25 conv2_filters = 6400
26 conv2_kernel_size = (10, 10, conv1_filters)  # 高 x 宽 x 深度
27 conv2_stride = 1
28 conv2_padding = 0
29 conv2_output_height = (pool_output_height + 2 * conv2_padding -
      conv2_kernel_size[0]) // conv2_stride + 1
30 conv2_output_width = (pool_output_width + 2 * conv2_padding -
      conv2_kernel_size[1]) // conv2_stride + 1
31 conv2_output_depth = conv2_filters
32 conv2_params = np.prod(conv2_kernel_size) * conv2_filters  # 卷
      积核参数量
33 conv2_bias = conv2_filters  # 每个卷积核有一个偏置
34 conv2_total_params = conv2_params + conv2_bias
35 # 4. 第四至六层: 全连接层
36 fc1_neurons = 1600
37 fc2_neurons = 400
38 fc3_neurons = 100
39 fc1_input = np.prod((conv2_output_height, conv2_output_width,
      conv2_output_depth))
40 fc1_params = fc1_input * fc1_neurons + fc1_neurons
41 fc2_params = fc1_neurons * fc2_neurons + fc2_neurons
42 fc3_params = fc2_neurons * fc3_neurons + fc3_neurons
43 # 总参数量
44 total_params = conv1_total_params + conv2_total_params +
      fc1_params + fc2_params + fc3_params
```

```
45 # 创建表格对象
46 table = PrettyTable()
47 # 设置表头
48 table.field_names = ["Layer", "Output/Neurons", "Parameters"]
49 # 添加层输出尺寸
50 table.add_row(["Conv1 Output", f"{conv1_output_height}x{
       conv1_output_width}x{conv1_output_depth}", f"{
       conv1_total_params}"])
51 table.add_row(["Pool Output", f"{pool_output_height}x{
       pool_output_width}x{pool_output_depth}", "N/A"])
52 table.add_row(["Conv2 Output", f"{conv2_output_height}x{
       conv2_output_width}x{conv2_output_depth}", f"{
       conv2_total_params}"])
53 table.add_row(["FC1 Neurons", f"{fc1_neurons}", f"{fc1_params}"
       ])
54 table.add_row(["FC2 Neurons", f"{fc2_neurons}", f"{fc2_params}"
       ])
55 table.add_row(["FC3 Neurons", f"{fc3_neurons}", f"{fc3_params}"
       ])
56 # 打印层输出尺寸和参数量
57 print("层输出尺寸与参数量: ")
58 print(table)
59 # 总参数量
60 print(f"\nTotal Parameters: {total_params}")
```